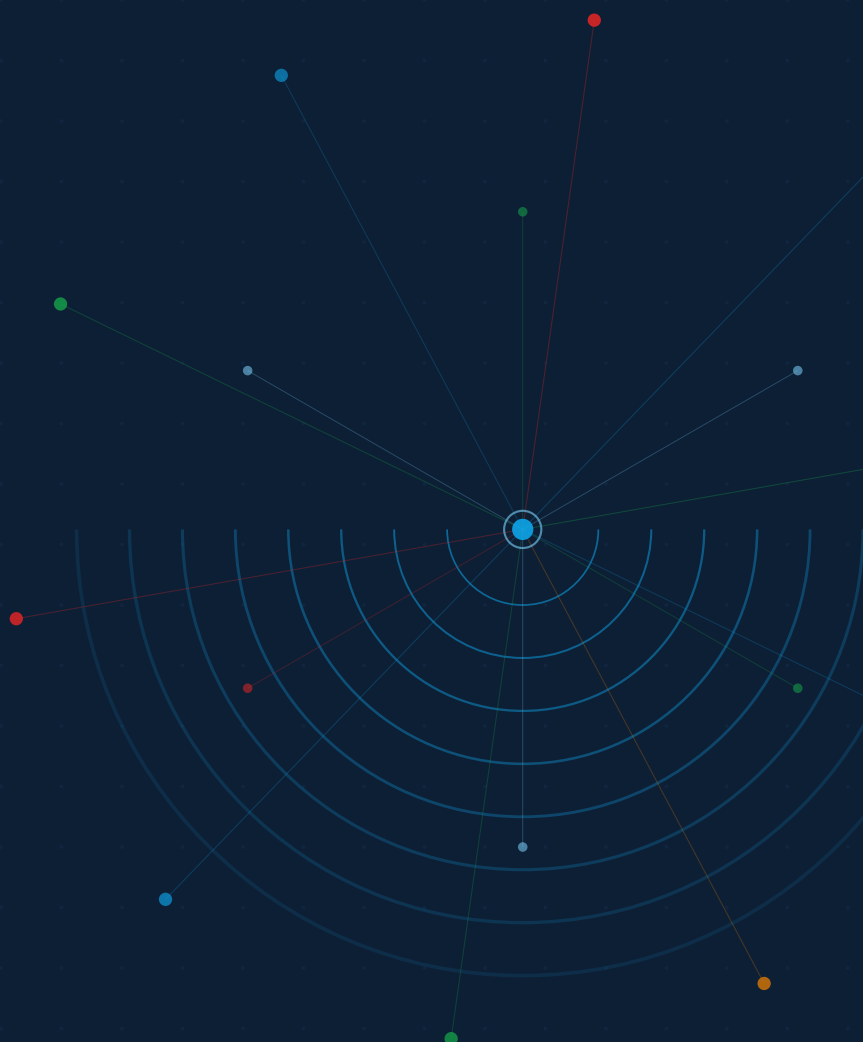


JA4proxy

Enterprise TLS Security Proxy

Product Overview & Buying Guide

For Security Buyers and Technical Evaluators



Stop the Bots That Look Human

A CREDENTIAL-STUFFING CAMPAIGN LOOKS IDENTICAL TO HUMAN LOGIN TRAFFIC at the HTTP layer. The requests are syntactically valid. The user-agent strings are plausible. Your CDN’s bot-score misses it because the attacker rotates IP addresses faster than reputation feeds update.

1

The usual answer — TLS inspection and decryption — is a poor trade. It adds latency, demands key custody, and creates a compliance exposure that PCI-DSS, HIPAA, and GDPR all penalise. JA4proxy takes the opposite approach: it makes its decision *before* decryption, reading only the plaintext the client volunteers in its ClientHello. It never holds a key, never decrypts a byte, and forwards allowed traffic through unchanged.

The JA4 Insight: Identity Before Encryption

BEFORE A SINGLE BYTE OF ENCRYPTED DATA IS EXCHANGED, every TLS client announces its capabilities in cleartext — TLS version, cipher suites, and extensions (SNI, ALPN). Because that announcement reflects the client’s TLS stack, the fingerprint is remarkably stable: Sliver 1.5.x produces the same ClientHello whether it runs from Frankfurt or Tokyo, and swapping the *payload* does nothing to change it. You cannot patch your way out of your own handshake.

Client	JA4 Fingerprint (example)	Disposition
Chrome 125 (Win)	t13d1516h2_8daaf6152771_02713d6af86	ALLOW
Firefox 126	t13d1915h2_cd87aab0d655_5d1a4de8bd1	ALLOW
Sliver C2	t13d2012_c1e4e3a9b26a_000000000000	BLOCK
Cobalt Strike	t13d11112_b4f47bba57b5_9b5e0e9c9d74	BLOCK

¹ The Verizon DBIR consistently shows that credential abuse accounts for more than 40% of breaches. Most credential-stuffing toolkits ship with TLS stacks that produce distinctive, stable fingerprints — yet traditional defences never inspect the handshake.

Table 1: Representative JA4 fingerprints. Browser fingerprints carry an h2 field; C2 frameworks typically do not.

Throughput, Not Trade-offs

JA4PROXY V2.0.0 IS A PURE GO-NATIVE ENGINE that sustains roughly **2,500 connections per second per node**² — enough to front a busy site from a single modest box, scaling linearly as you add nodes.

That headroom comes from a deliberately small core: a fingerprint decision on the allow path costs about **272 ns**, so the proxy adds latency your users will never feel. Layer-by-layer isolation testing proved the security pipeline itself is never the bottleneck — your network is.

² Measured end-to-end on commodity hardware (Intel i9-9900K) with host networking and the full scoring pipeline active — zero errors, zero false positives. Bridged container networking is lower; deploy with host networking for line-rate throughput.

Decision path	Core latency	Heap allocs
Allowed (Fast Path)	272 ns	1 object
Scored (Standard Path)	2.3 μs	9 objects
Adversarial (JA4X)	6.2 μs	73 objects

Table 2: Per-connection pipeline cost, isolated from network I/O (Intel i9-9900K). A single static binary, under 12 MB resident.

It Cannot Block Your Real Customers

THE EXPENSIVE FAILURE FOR A SECURITY PROXY IS A FALSE POSITIVE — a real browser turned away. JA4proxy is built so that outcome is structurally hard to reach.

- **Monitor-mode by default.** Out of the box the dial sits at `dial=0` (monitor only): every connection is scored and logged, nothing is blocked. You raise the dial only once you trust the scores.
- **Browser traffic bypasses everything.** `h2/h1` ALPN handshakes are allowed before the scorer ever runs — a real browser cannot be blocked, by design.
- **Fail open, always.** If Redis or any enrichment service is unreachable, JA4proxy allows and logs. A degraded dependency never becomes an outage for your users.

3

Deploys in Minutes

THE ENGINE IS ONE STATICALLY-LINKED GO BINARY in a minimal container image — no runtime, no interpreter, nothing to patch underneath it. Bring up the bundled Docker Compose stack, or run the container under the orchestrator of your choice, and `ja4pd` starts in milliseconds, ready for auto-scaling DMZ deployments. The whole project is open source: clone the `v2.0.0` release and `make build` the engine and its companion CLI (`ja4p`) from source whenever you prefer.

For high-traffic deployments the engine adds integrated cluster synchronisation, Zero-Trust Redis hardening over TLS, and PROXY protocol v2 with TLV-based trust verification. Real-time Prometheus and Grafana metrics give you granular visibility into the full security waterfall — without costing you throughput.

Source code and technical evaluators' guide:
<https://github.com/seanpor/JA4proxy>
JA4 Specification: <https://github.com/FoxIO-LLC/ja4>

³ No decryption means no key custody, no certificate logistics, and no new audit scope. JA4proxy is transparent to both client and backend, and forwards allowed connections byte-for-byte.

What JA4proxy does

JA4proxy intercepts the TLS ClientHello — which is always transmitted in plaintext, before encryption begins. It computes a JA4 fingerprint from the cipher suites, extensions, ALPN, and elliptic curves advertised by each client, then routes the connection through a multi-signal scoring pipeline to decide: *allow, tarpit, block, or ban*. Allowed connections are forwarded byte-for-byte unchanged. The proxy never decrypts, never holds your private keys, and is transparent to both client and backend.

Key facts

- 0% false positive rate on browser traffic — by design
- 94–99% of malicious traffic blocked in testing
- Default dial = 0: monitor mode, never blocks on first deploy
- All bans carry a 5-minute TTL — false positives self-heal
- Never decrypts: no key custody, no SSL inspection compliance risk
- Scales horizontally; Redis synchronises bans across all instances

Resources

Source code & issue tracker: github.com/seanpor/JA4proxy

JA4 fingerprint specification: github.com/FoxIO-LLC/ja4

Documentation index: `docs/INDEX.md` in the repository